GROUPE 4 · AI-BASED APPLICATIONS · SoSe 2026

Chat with a PDF

A RAG assistant for exploring corporate sustainability reports

Team

Schalum Semenyo
Zahidul Alam
Amin Touati

Dr. Debayan Banerjee · Prof. Dr. Ricardo Usbeck · Leuphana University Lüneburg

The Task

Chat with a sustainability report

- Companies publish environmental sustainability reports as dense PDFs
- Our web app lets a user upload one of these reports and ask questions in plain language
- The system retrieves the relevant passages and lets an LLM answer using only that context
- A structured JSON report (CO2, NOX, EV count, risks, targets, …) can be generated and downloaded

Example question

“How did carbon emissions change between 2024 and 2025?”

The app retrieves the matching passages, cites the page number, and answers directly from the source.

Data source: saturnMars / derivingStructuredInsightsFromSustainabilityReports (arXiv 2310.05628)

Team

Individual contributions

Schalum Semenyo

PROJECT MANAGEMENT & REPO

- GitHub maintenance & repo clean-up
- README
- Trello / SCRUM board upkeep

Zahidul Alam

RAG PIPELINE

- rag/ingest.py — PDF extraction, chunking, embeddings, FAISS
- rag/query.py — retrieval + GWDG LLM calls

Amin Touati

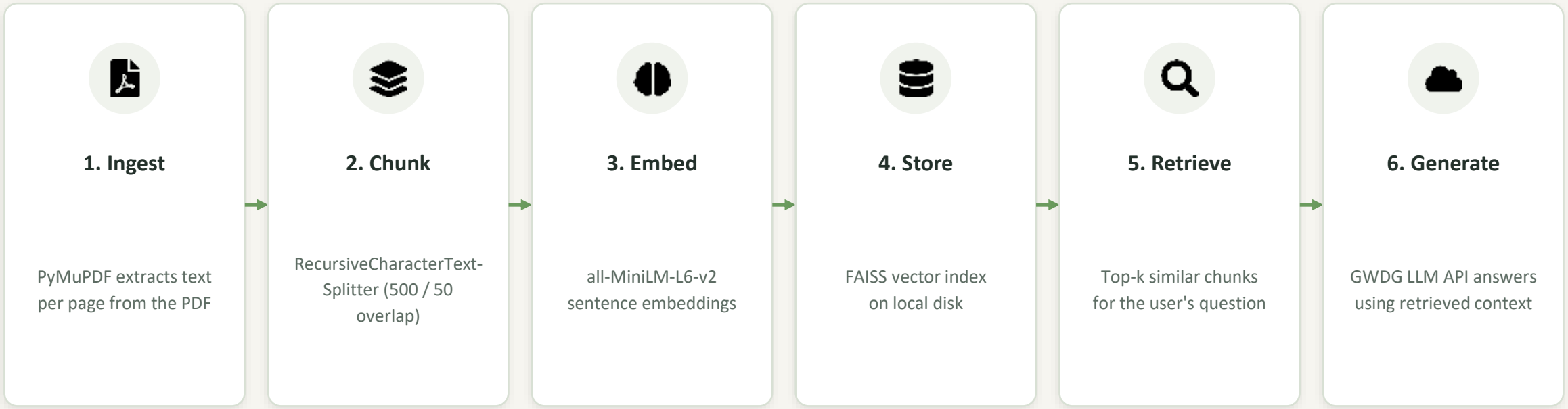
APPLICATION & UX

- Streamlit UI build-out
- Feedback & source-highlighting features
- JSON report extraction

The whole team can explain any part of the code during Q&A.

Architecture

The RAG pipeline



Runs entirely on the laptop — only the LLM call goes out to the GWDG-hosted Academic Cloud API.

Frameworks & choices

Tech Stack

Streamlit

FRONTEND + APP

Fastest way to ship a working Python UI without writing separate frontend/backend code

PyMuPDF

PDF EXTRACTION

Reliable, fast text extraction that preserves page numbers for citations

LangChain

CHUNKING

RecursiveCharacterTextSplitter handles chunk size / overlap out of the box

all-MiniLM-L6-v2

EMBEDDINGS

Small, fast, runs locally on a laptop — no external embedding API needed

FAISS

VECTOR STORE

Local, in-memory similarity search recommended for speed in the course

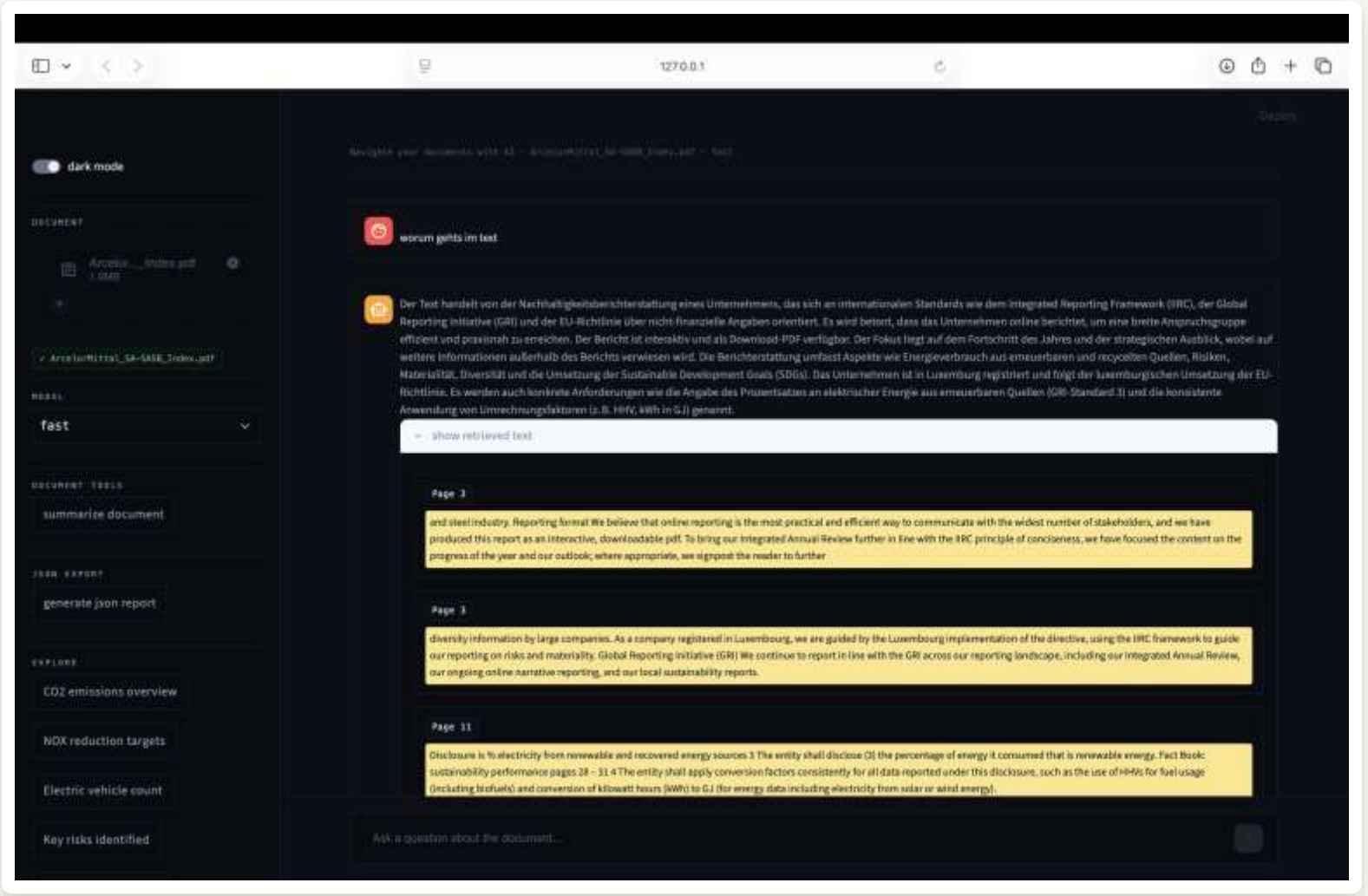
GWDG API

LLM (QWEN3-30B)

Free university-hosted, OpenAI-compatible endpoint for the answer generation

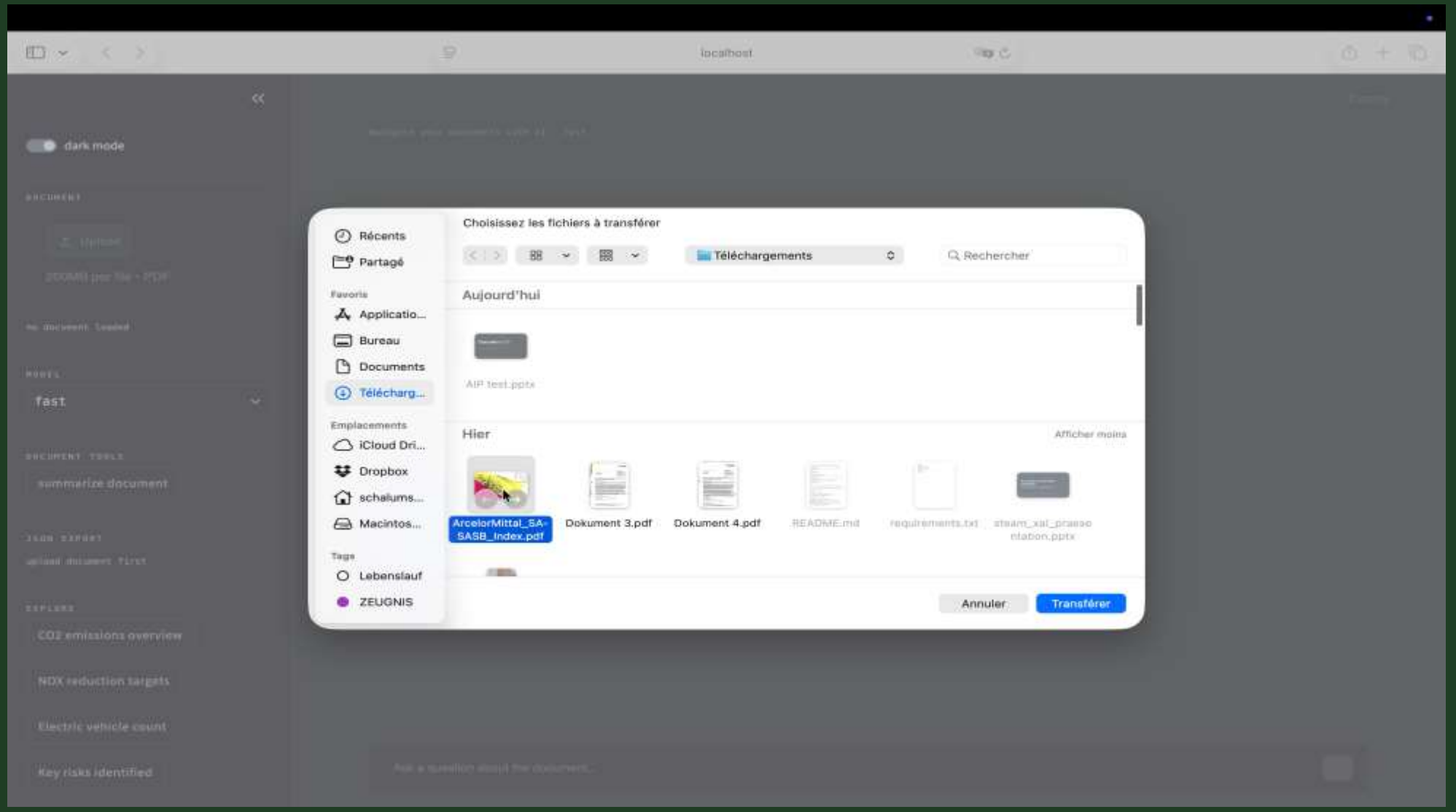
Live Demo

Ask a question, get a cited answer



Live Demo

Full walkthrough



Retrospective

What worked, what didn't

Worked well

- End-to-end RAG pipeline runs reliably on a laptop
- Retrieved passages shown with page numbers for transparency
- Structured JSON export automates report extraction
- Docker setup makes the app portable
- GitHub + Trello kept the team in sync

Challenges

- Coordinating parallel work on the RAG core across teammates
- Getting comfortable with Git push/pull as a team new to version control
- Balancing chunk size for speed vs. answer accuracy
- Missing error handling early on caused raw crashes — fixed with try/except across the app
- Tight timeline between first working version and final polish

Vibe coding

AI tools we used

Claude (Anthropic)

Used throughout the project for: drafting the RAG pipeline (rag/ingest.py, rag/query.py), reviewing and hardening the Streamlit UI (error handling, input escaping) and project-management guidance accessed via claude.ai and Claude Code (JetBrains plugin in PyCharm).

Every teammate reviewed and understood the AI-suggested code before committing it — we can walk through any part of the pipeline in the Q&A.

Thank you

Questions & live walkthrough

GitHub

github.com/Schalum/Groupe4AIApplication-

Trello

trello.com/b/wOgfu3P3/ai-based-application